

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – CODING CHALLENGE

|                  |                                                                                                                                                          |               |                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|--------------------------------------|
| Course Code:     | DSA03                                                                                                                                                    | Course Title: | Natural Language Processing          |
| CO Assessed:     | CO1 – Imitation (L1)                                                                                                                                     | Assessment:   | Assessment Tool 3 – Coding Challenge |
| Weightage:       | 30% of CIA                                                                                                                                               |               |                                      |
| CO5 Statement:   | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) |               |                                      |
| Student Name:    | N. Hemanth Reddy                                                                                                                                         | Reg. No.:     | 192472215                            |
| Section / Batch: | SLOT A / 2024                                                                                                                                            | Date:         | 27/07/2026                           |

### Code of Conduct

*I\_N. Hemanth Reddy (192472215)\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: \_Hemanth Reddy\_

### Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

| Section / Topic                      | Focus                                                  | Q Nos |
|--------------------------------------|--------------------------------------------------------|-------|
| Regular Expression Pattern Matching  | Regular Expressions, Basic Regular Expression Patterns | 1     |
| Finite State Automata Implementation | Finite State Automata, Models and Algorithms           | 2     |
| Advanced Regular Expression Search   | Regular Expressions, Basic Regular Expression Patterns | 3     |

### Section 1: Intelligent Email and Password Validator using Regular Expressions

#### 1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

#### Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

#### Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '\_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

#### Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

#### Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

#### Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

### Code

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Email Validation
email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"

# Password Validation
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$"

# Mobile Validation
mobile_pattern = r"^[6-9]\d{9}$"

if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
```

```
if re.match(password_pattern, password):
```

```
    print("Strong Password")
```

```
else:
```

```
    print("Weak Password")
```

```
if re.match(mobile_pattern, mobile):
```

```
    print("Valid Mobile Number")
```

```
else:
```

```
    print("Invalid Mobile Number")
```

### Sample Input:

**Enter Email:** hemureddy@gmail.com

**Enter Password:** Hemu@2005

**Enter Mobile Number:** 9490317331

### Sample Output:

**Valid Email**

**Strong Password**

**Valid Mobile Number**

## Section 2: Design a Finite State Automata (FSA) Simulator

### 2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

**The program should:**

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

#### Example

Language:  
Strings ending with "ab"

**Input**  
abaab

**Output**  
Transition Path:  
 $q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$   
Accepted

[Code:](#)

```
def dfa(string):
    state = "q0"
    path = [state]

    for ch in string:
        if state == "q0":
            if ch == "a":
                state = "q1"
            else:
                state = "q0"

        elif state == "q1":
            if ch == "a":
                state = "q1"
            elif ch == "b":
                state = "q2"
            else:
                state = "q0"

        elif state == "q2":
            if ch == "a":
                state = "q1"
            else:
                state = "q0"

    path.append(state)

    print("Transition Path:")
    print(" -> ".join(path))

    if state == "q2":
        print("Accepted")
    else:
        print("Rejected")
```

```
text = input("Enter String: ")
dfa(text)
```

### Sample Input:

Enter String: *abaab*

### Sample Output:

**Transition Path:**

**q0 -> q1 -> q1 -> q1 -> q1 -> q2**

**Accepted**

### 3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

#### The system should support:

Word Search  
Prefix Search  
Suffix Search  
Date Search  
Phone Number Search  
Hashtag Search  
Mention (@username) Search

#### Example Input

Meeting on 12/09/2026  
Call 9876543210  
#NLP  
@OpenAI  
natural language processing

#### User Menu

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

#### Expected Output

Display all matching patterns based on user selection.

#### Code:

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

print("1.Date")
print("2.Phone")
print("3.Hashtag")
print("4.Mention")
print("5.Prefix")
print("6.Suffix")

choice = int(input("Enter Choice: "))

if choice == 1:
    print(re.findall(r"\d{2}\d{2}\d{4}", text))

elif choice == 2:
    print(re.findall(r"[6-9]\d{9}", text))
```

```
elif choice == 3:
    print(re.findall(r"#\w+", text))

elif choice == 4:
    print(re.findall(r"@ \w+", text))

elif choice == 5:
    prefix = input("Enter Prefix: ")
    print(re.findall(r"\b" + prefix + r"\w*", text))

elif choice == 6:
    suffix = input("Enter Suffix: ")
    print(re.findall(r"\b\w*" + suffix + r"\b", text))
```

### Sample Input:

- 1.Date
- 2.Phone
- 3.Hashtag
- 4.Mention
- 5.Prefix
- 6.Suffix

### Sample Output:

Enter Choice: 3  
[#NLP]

## Rubrics for Calculation

| Criteria                       | Weightage | Level 4 – Exemplary                                                                       | Level 3 – Proficient                                      | Level 2 – Developing                          | Level 1 – Beginning                              |
|--------------------------------|-----------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|
| <b>Problem Understanding</b>   | 20%       | Clearly understands problem constraints, edge cases, and competitive requirements         | Understands problem with minor gaps in constraints        | Partial understanding; misses key constraints | Misinterprets problem or constraints             |
| <b>Algorithm</b>               | 25%       | Selects optimal algorithm with best time and space complexity for competitive constraints | Uses efficient algorithm with minor scope for improvement | Uses basic/inefficient approach               | No clear algorithmic approach                    |
| <b>Code Implementation</b>     | 20%       | Writes correct, efficient, and clean code that passes all constraints                     | Code works with minor errors or inefficiencies            | Partially correct code; fails for some cases  | Code does not execute or gives incorrect results |
| <b>Competitive Performance</b> | 20%       | Solves problem within time limits and handles large inputs effectively                    | Solves within acceptable time with minor delays           | Struggles with time limits or larger inputs   | Unable to solve within time constraints          |
| <b>Test Case Handling</b>      | 15%       | Handles all test cases including edge and hidden cases                                    | Handles most test cases                                   | Misses important edge cases                   | Fails on multiple test cases                     |

| Q. No. / Criteria | Problem Understanding | Algorithm | Code Implementation | Competitive Performance | Test Case Handling | Sub. Total | Total % |
|-------------------|-----------------------|-----------|---------------------|-------------------------|--------------------|------------|---------|
| 1                 |                       |           |                     |                         |                    |            |         |
| 2                 |                       |           |                     |                         |                    |            |         |
| 3                 |                       |           |                     |                         |                    |            |         |

| Faculty In-charge | Course Coordinator | Program Director |
|-------------------|--------------------|------------------|
|                   |                    |                  |
| Signature: _____  | Signature: _____   | Signature: _____ |
| Date: _____       | Date: _____        | Date: _____      |